

# A Parallel Approach to Solve the Approximation String Matching Problem

C. S. Ou and R. C. T. Lee

Department of Computer Science

National Tsing Hua University

d9662836@oz.nthu.edu.tw and rctlee@ncnu.edu.tw

## Abstract

In this paper, we present a new parallel technique for approximation string matching problem. Our idea is based upon on Myers' Algorithm[10] which searches for a pattern of length  $m$  in a text of length  $n$  permitting edit distance equal to  $k$  in  $O(\lceil m/w \rceil n)$  time, where  $w$  is the word size of computer. And, we use a parallel approach to avoid unnecessary operations by the diagonal property[4].

## 1 Introduction

In this paper, we consider the following problem: Given a text  $T = t_1 t_2 \cdots t_n$ , a pattern  $P = p_1 p_2 \cdots p_m$  and an error bound  $k$ , we are asked to find whether there exists a prefix of  $T$  whose edit distance with  $P$  is smaller than or equal to  $k$ . The edit distance between  $A$  and  $B$  is the minimum number of insertion, deletion and substitution operations needed to transform  $B$  into  $A$ [4]. This problem can be computed by using dynamic programming method [5, 14]. Many approximate string matching algorithms employ this dynamic programming approach [1,2,3,6,7,8,9,10,11,13,14,15,16,17,18]. Let  $c(i, j)$  denote the edit distance between  $T(1, i)$  and  $P(1, j)$ . Then  $c(i, j)$  can be computed by the following formulas:

### Equation 1.1

$$\begin{cases} c_{i,0}=i, & c_{0,j}=j, \\ c_{i,j}=c_{i-1,j-1} & \text{if } (t_i=p_j) \text{ else } 1+\min(c_{i-1,j}, c_{i,j-1}, c_{i-1,j-1}), \end{cases}$$

Consider  $T = ggtac$  and  $P = acggt$ . We can use Equation 1.1 to construct the dynamic programming matrix shown in Table 1.

Table 1 Dynamic Programming Matrix of  $T = ggtac$  and  $P = acggt$

|   |     |     |   |   |   |   |   |
|---|-----|-----|---|---|---|---|---|
|   |     |     | 1 | 2 | 3 | 4 | 5 |
|   |     | $T$ | g | g | t | c | a |
|   | $P$ | 0   | 1 | 2 | 3 | 4 | 5 |
| 1 | a   | 1   | 1 | 2 | 3 | 4 | 4 |
| 2 | c   | 2   | 2 | 2 | 3 | 4 | 4 |
| 3 | g   | 3   | 2 | 2 | 3 | 4 | 4 |
| 4 | g   | 4   | 3 | 2 | 3 | 4 | 5 |
| 5 | t   | 5   | 4 | 3 | 2 | 3 | 4 |

From the above table, we know that the edit distance between  $P(1,3)$  and  $T$  is 2. If we specify the error bound to be  $k = 2$ ,  $P(1,3)$  is the only solution.

In this paper, we are interested in mechanisms to compute the above table efficiently. Our approach is based upon the Myer's algorithm [9] which was designed to solve the following type of approximate string matching problem : Given a text string  $T$  and a pattern string  $P$ , for each location  $i$ , determine whether there exists a substring ending at location  $i$  whose edit distance with  $P$  is less than or equal to a presepified error bound  $k$ . Myer's algorithm was further modified in [9] to solve the following approximate string matching problem: Given a text string  $T$  and a pattern string  $P$ , find all locations  $i$  where some substring starting from  $i$  has edit distance with  $P$  is less than or equal to a presepified error bound  $k$ . In the following, we shall introduce the basic approach of Myer's algorithm.

## 2 A Brief Introduction of the Myer's Algorithm

To understand Myer's Algorithm[10], we need to introduce some notations.

### Definition 2.1

Let  $c(i, j)$  denote the value of location  $(i, j)$  of the dynamic programming matrix.

$$C(i) = (c(i,0), c(i,1), \dots, c(i,m))$$

For instance, in Table 2.1,  $C(2) = (2, 2, 2, 2, 3)$  and  $C(4) = (4, 3, 4, 4, 3)$ .

**Definition 2.2**

$$\Delta h(i, j) = c(i, j) - c(i-1, j)$$

$$\Delta v(i, j) = c(i, j) - c(i, j-1)$$

For example, in Table

$$2, \Delta h(2,3) = c(2,3) - c(1,3) = 2 - 2 = 0 \quad \text{and}$$

$$\Delta v(2,5) = c(2,5) - c(2,4) = 3 - 2 = 1.$$

**Definition 2.3**

$$\Delta H(i) = (\Delta h(i,0), \Delta h(i,1), \dots, \Delta h(i,m))$$

$$\Delta V(i) = (\Delta v(i,0), \Delta v(i,1), \dots, \Delta v(i,n))$$

For example, in Table 2.1,

$$\Delta H(2) = (2 - 1, 2 - 1, 2 - 2, 2 - 2, 3 - 4)$$

$$= (1, 1, 0, 0, -1).$$
 And,

$$\Delta V(2) = (2 - 2, 2 - 2, 2 - 2, 2 - 2, 3 - 2)$$

$$= (0, 0, 0, 0, 1).$$

Based upon Definitions 2.1 and 2.2, we may redraw Table 1 into Table 2 as follows:

Table 2 Redraw Table 1.1 by Definition 2.1 and 2.2

|   |     | $T$ | $1$ | $2$ | $3$ | $4$ | $5$ |
|---|-----|-----|-----|-----|-----|-----|-----|
|   |     | $g$ | $g$ | $t$ | $c$ | $a$ |     |
|   | $P$ | 0   | 1   | 2   | 3   | 4   | 5   |
| 1 | $a$ | 1   | 0   | 1   | 2   | 3   | 4   |
| 2 | $c$ | 2   | 0   | 2   | 0   | 3   | 1   |
| 3 | $g$ | 3   | -1  | 2   | 0   | 4   | 0   |
| 4 | $g$ | 4   | -1  | 3   | -1  | 2   | 1   |
| 5 | $t$ | 5   | -1  | 4   | -1  | 3   | -1  |

We know that  $c(i, j)$  is determined by its three neighbors:  $c(i, j-1)$ ,  $c(i-1, j)$  and  $c(i-1, j-1)$ , as shown in Figure 1.

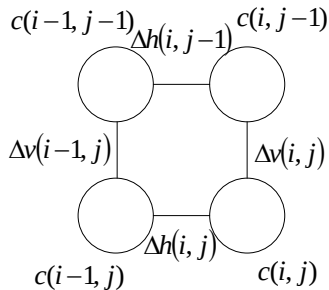


Figure 1  $c(i, j)$  with its three neighbors:

$$c(i, j-1), c(i-1, j) \quad \text{and} \quad c(i-1, j-1)$$

Let us introduce the following Boolean variables.

**Definition 2.4**

$$Pv(i, j) \equiv (\Delta v(i, j) = +1)$$

$$Mv(i, j) \equiv (\Delta v(i, j) = -1)$$

$$Ph(i, j) \equiv (\Delta h(i, j) = +1)$$

$$Mh(i, j) \equiv (\Delta h(i, j) = -1)$$

For instance, in Table 2.1,  $\Delta v(2,5) = 1$ ,  $\Delta h(2,3) = 0$ ,  $Pv(2,5) = 1$ ,  $Mv(2,5) = 0$ ,  $Ph(2,3) = 0$  and  $Mh(2,3) = 0$ .

**Definition 2.5**

$eq(i, j) = 1$  if  $t_i = p_j$  and  $eq(i, j) = 0$  if otherwise.

For instance, in Table 2.1,  $t_3 = p_5 = t$  and  $eq(3,5) = 0$ .

Based upon the above discussion, we define the following seven column vectors:

**Definition 2.6**

$$PV(i) = (Pv(i,1), Pv(i,2), \dots, Pv(i,m))$$

$$MV(i) = (Mv(i,1), Mv(i,2), \dots, Mv(i,m))$$

$$PH_1(i) = (Ph(i,1), Ph(i,2), \dots, Ph(i,m))$$

$$PH_2(i) = (Ph(i,0), Ph(i,1), \dots, Ph(i, m-1))$$

$$MH_1(i) = (Mh(i,1), Mh(i,2), \dots, Mh(i,m))$$

$$MH_2(i) = (Mh(i,0), Mh(i,1), \dots, Mh(i, m-1))$$

$$EQ(i) = (eq(i,1), eq(i,2), \dots, eq(i,m))$$

For instance, in Table 2.1, the following are all true:

$$Pv(2,1) = 0, \quad Pv(2,2) = 0, \quad Pv(2,3) = 0,$$

$$Pv(2,4) = 0, \quad Pv(2,5) = 1,$$

$$PV(2) = (Pv(2,1), Pv(2,2), Pv(2,3), Pv(2,4), Pv(2,5)) = (0, 0, 0, 0, 1).$$

$$Mv(2,1) = 0, \quad Mv(2,2) = 0, \quad Mv(2,3) = 0,$$

$$Mv(2,4) = 0, \quad Mv(2,5) = 0.$$

$$MV(2) = (Mv(2,1), Mv(2,2), Mv(2,3), Mv(2,4), Mv(2,5)) = (0, 0, 0, 0, 0).$$

$$Ph(2,0) = 1, \quad Ph(2,1) = 1, \quad Ph(2,2) = 1,$$

$$Ph(2,3) = 1, \quad Ph(2,4) = 1 \quad \text{and} \quad Ph(2,5) = 1.$$

$$PH_1(2) = (Ph(2,1), Ph(2,2), Ph(2,3), Ph(2,4), Ph(2,5)) = (1, 1, 1, 1, 0).$$

$$PH_2(2) = (Ph(2,0), Ph(2,1), Ph(2,2), Ph(2,3), Ph(2,4)) = (1, 1, 1, 1, 1).$$

$$Mh(2,0) = 0, \quad Mh(2,1) = 0, \quad Mh(2,2) = 0,$$

$Mh(2,3)=0$ ,  $Mh(2,4)=0$  and  $Mh(2,5)=1$ .

$MH_1(2) = (Mh(2,1), Mh(2,2), Mh(2,3), Mh(2,4), Mh(2,5))$   
 $= (0,0,0,0,1)$ .

$MH_2(2) = (Mh(2,0), Mh(2,1), Mh(2,2), Mh(2,3), Mh(2,4))$   
 $= (0,0,0,0,0)$ .

According to Myer's Algorithm[10], we can obtain the following logical formulas:

**Lemma 1**

$PV(i) = MH_2(i)$  or  $(\neg EQ(i) \text{ and } \neg MV(i-1) \text{ and } \neg PH_2(i))$

$MV(i) = PH_2(i)$  and  $(EQ(i) \text{ or } MV(i-1))$

$PH_1(i) = MV(i-1)$  or  $(\neg EQ(i) \text{ and } \neg MH_2(i) \text{ and } \neg PV(i-1))$

$MH_1(i) = PV(i-1)$  and  $(EQ(i) \text{ or } MH_2(i))$

For our problem, we have  $C(0) = (1, 2, \dots, m)$  as an initial vector to start with. After computing  $C(i-1)$ ,  $PH_1(i)$  and  $MH_1(i)$ , we can obtain  $C(i)$ . From the above formulas, we can see that the dynamic programming table can be obtained in a bit-parallel way. It is a question whether we need to obtain the whole dynamic programming table or not. Assume that  $k = 2$ . Let us take a look at Table 2. We can easily see that after Column 3, no  $c(i, j) \leq k = 2$  appears. To explain this phenomenon, we need to understand the so-called diagonal property of dynamic programming. Let us redraw Table 2 as follows:

Table 3 Redraw Table 2

|          |   |          |   |   |   |   |   |
|----------|---|----------|---|---|---|---|---|
|          |   | <b>T</b> | 1 | 2 | 3 | 4 | 5 |
|          |   |          | g | g | t | c | a |
| <b>P</b> | 0 | 1        | 2 | 3 | 4 | 5 |   |
| 1 a      | 1 | 1        | 2 | 3 | 4 | 4 |   |
| 2 c      | 2 | 2        | 2 | 3 | 4 | 4 |   |
| 3 g      | 3 | 2        | 2 | 3 | 4 | 4 |   |
| 4 g      | 4 | 3        | 2 | 3 | 4 | 5 |   |
| 5 t      | 5 | 4        | 3 | 2 | 3 | 4 |   |

A diagonal  $d$  of a dynamic programming table consists all of  $c(i, j)$ 's where  $i - j = d$  where  $d$  is a constant. In Table 2.2, we have shown three diagonals, namely Diagonals -1, 0 and 1. It can be proved that the values on any diagonal are non-decreasing [4]. This special property is called the diagonal property which was used in the

LV Algorithm [3] and the TU Algorithm [15]. The HN Algorithm [9] also uses this property to ignore some unnecessary computations.

Based upon the diagonal property, we can easily see that if in a column  $i$ , all values in the column, except the last location, are greater than  $k$ , all columns after column  $i$  do not have to be computed. We still have a problem: How can we know that in a column  $i$ , all values in the column, except the last location, are greater than  $k$ ?

### 3 The Termination of the Dynamic Programming Computing

As we discussed in the above, our dynamic programming can be terminated if all values of are greater than  $k$ . To check this, we define a term  $ex = 2^{\lceil \log_2(m+k) \rceil - 1} - 1 - k$ . For example,  $m = 6$  and  $k = 2$ . Then,  $ex = 2^{\lceil \log_2(6+2) \rceil - 1} - 1 - 2 = 2^{\log_2 8 - 1} - 3 = 2^{3-1} - 3 = 1$ . We further define a term  $ce_j = c(i, j-1) + ex$ . For instance,  $c(i, j-1) = 2$ . Then  $ce_j = 2 + 1 = 3$ . Let  $bce_j$  denote the binary representation of  $ce_j$ . For instance, if  $ce_j = 3$ ,  $bce_j = 011$  if we stipulate that  $bce_j$  holds 3 bits. Finally, we use a vector  $bc_j$  with size  $\lceil \log_2(m+k) \rceil$  consisting of all of the bits of  $bce_j$ . For instance, if  $m = 6$  and  $k = 2$ ,  $\lceil \log_2(m+k) \rceil = \lceil \log_2(6+2) \rceil = 3$ . Thus, vector  $bc_j = (0,1,1)$  in the above case. Besides,  $bc_j(1) = 0$ ,  $bc_j(2) = 1$  and  $bc_j(3) = 1$ . Finally, we define a matrix  $BC(m, \lceil \log_2(m+k) \rceil) = (bc_1, bc_2, \dots, bc_m)$ . For instance, after Column 4 of Table is computed, we have  $C(1) = (1,1,2,2,3,4)$ . Assume that  $ex = 1$  then  $ce_1 = 2$ ,  $ce_2 = 2$ ,  $ce_3 = 3$ ,  $ce_4 = 3$  and  $ce_5 = 4$ . Then,  $bce_1 = 010$ ,  $bce_2 = 010$ ,  $bce_3 = 011$ ,  $bce_4 = 011$  and  $bce_5 = 100$ . We will have the following BC matrix:

Table 3.1 BC table of  $C(1) = (1,1,2,2,3,4)$

$bc_j(i)$

| i \ j | 1 | 2 | 3 |
|-------|---|---|---|
| 1     | 0 | 1 | 0 |

|   |   |   |   |
|---|---|---|---|
| 2 | 0 | 1 | 0 |
| 3 | 0 | 1 | 1 |
| 4 | 0 | 1 | 1 |
| 5 | 1 | 0 | 0 |

The following property of the BC matrix can be easily seen:

**Property 1**

$bc_j(1) = 1$  if and only if  $c(i, j-1) > k$ .

To explain this property, assume that  $c(i, j-1) = x$ . Then  $bc_j = x + ex$ . Hence, if  $c(i, j) > k$ ,  $bc_j = x + ex > k + 2^{\lceil \log_2(m+k) \rceil - 1} - k + 1 = 2^{\lceil \log_2(m+k) \rceil - 1} + 1$ . When  $bc_j > 2^{\lceil \log_2(m+k) \rceil - 1} + 1$ , we know the  $\lceil \log_2(m+k) \rceil$ -th bit of  $bc_j$  must be 1. For instance, given  $c(2,5) = 3$ ,  $m = 6$  and  $k = 2$ , for  $C(1) = (1,1,2,2,3,4)$  and  $ex = 1$ , we have  $ce_5 = 4$ . We get the size of  $bc_5 = \lceil \log_2(6+2) \rceil = 3$ . Hence,  $bce_5 = 100$  and  $bc_5 = (1,0,0)$ . Because  $bc_5(1) = 1$ ,  $c(1,4) > 2$ .

From this property, we know that  $c(i, j-1) > k$  if and only if  $bc_j(1) = 1$  for  $1 \leq j \leq m$ . The problem is: How can we determine that  $bc_j(1) = 1$  for  $1 \leq j \leq m$  in the bit-parallel fashion. This will be explained in the following section.

## 4 The For All Checking Problem

In order to solve  $bc_j(1) = 1$  for  $1 \leq j \leq m$ , we define the following problem: Given  $A = (a_1, a_2, \dots, a_n)$  and  $j$ , we want to find a vector  $C$  such that  $C(k) = 1$  if and only if  $A(j) = A(j+1) = \dots = A(k) = 1$ . We denote Vector  $A$  as the main vector of the problem and define an auxiliary vector, namely Vector  $B = (0, 0, \dots, b_j = 1, 0, \dots, 0)$  for Vector  $A$ . In [12], we proved that we can use the following equation

to solve the For All Checking problem.

**Equation 4.1**

$$C \equiv (((B + A) \text{ xor } A) \text{ and } A)$$

Here, the operation  $+$  is the binary addition from left to right.

For instance,  $A = (0, 0, 1, 1, 1, 0)$  and  $j = 3$ .

$$\begin{aligned} C &\equiv (((B + A) \text{ xor } A) \text{ and } A) \\ &= (((0, 0, 1, 0, 0, 0) + (0, 0, 1, 1, 1, 0)) \text{ xor } (0, 0, 1, 1, 1, 0)) \\ &\quad \text{and } (0, 0, 1, 1, 1, 0)) \\ &= (((0, 0, 0, 0, 0, 1) \text{ xor } (0, 0, 1, 1, 1, 0)) \text{ and } (0, 0, 1, 1, 1, 0)) \\ &= (0, 0, 1, 1, 1, 1) \text{ and } (0, 0, 1, 1, 1, 0) \\ &= (0, 0, 1, 1, 1, 0) \end{aligned}$$

We check whether  $C(k) = 1$  or not for  $k \geq 3$ . Since  $C(k) = 1$  for  $k = 3$  to  $k = 6$ , we conclude that  $A(i) = 0$  for  $3 \leq i \leq 6$ .

According to the For All Checking Problem, we can solve  $bc_j(1) = 1$  for  $1 \leq j \leq m$  by using the auxiliary vector  $B = (1, 0, \dots, 0)$ .  $C(m) = 1$  if and only if  $bc_j(1) = 1$  for  $1 \leq j \leq m$ .

Let  $CBC = (bc_1(1), bc_2(1), \dots, bc_m(1))$ . We use the following Equation to check the bits of  $CBC$  are 1.

**Equation 4.2**

$$(((CBC + 10^{m-1}) \text{ xor } CBC) \text{ and } CBC)$$

According to Equation 4.2, we know  $(((CBC + 10^{m-1}) \text{ xor } CBC) \text{ and } CBC)(m) = 1$  if and only if  $bc_j(1) = 1$  for  $1 \leq j \leq m$ . For instance,  $CBC = (1, 1, 1, 1, 1)$ ,  $((1, 1, 1, 1, 1) + (1, 0, 0, 0, 0) \text{ xor } (1, 1, 1, 1, 1) \text{ and } (1, 1, 1, 1, 1))(5) = 1$  and  $bc_j(1) = 1$  for  $1 \leq j \leq m$ . For instance,  $CBC = (1, 1, 0, 1, 1)$ ,

$((1,1,0,1,1) + (1,0,0,0,0) \text{ xor } (1,1,0,1,1) \text{ and } (1,1,0,1,1))(1) = (0,0,0,1,1)$  and  $bc_3(1) = 1$ .

There is another problem which is how to update  $BC$  matrix when we read the next character of  $W$ . We can use the following Equation to update  $BC$  matrix.

**Equation 4.3**

For column  $i$ ,  $bc_j = bc_j + ph(i, j-1) - mh(i, j-1)$  for  $1 \leq j \leq m$ .

For instance,  $C(4) = (4,4,4,4,3)$  and  $k = 2$ , we get  $bc_4 = (1,0,1)$ . And  $ph(5,5) = 0$  and  $mh(5,5) = 1$ . We update  $bc_4 = (1,0,1) + 0 - 1 = (1,0,0)$ .

Here, we define the operations between a matrix and a vector as follows:

**Definition 4.1**

Let  $MA = (M_1, M_2, \dots, M_n)$  be a matrix where  $M_i$  is a vector,  $1 \leq i \leq n$  and  $V = (v_1, v_2, \dots, v_n)$  be a vector where every  $v_i$  is a bit.  
 $MA + V = (M_1 + v_1, M_2 + v_2, \dots, M_n + v_n)$  and  
 $MA - V = (M_1 - v_1, M_2 - v_2, \dots, M_n - v_n)$ .

For instance,  
 $MA = ((1,1,0,1), (0,0,1,1), (1,0,0,0), (1,1,1,1))$  and  
 $V = (1,0,1,0)$   
 $MA + V = ((1,1,0,1) + 1, (0,0,1,1) + 0, (1,0,0,0) + 1, (1,1,1,1) + 0)$   
 $= ((1,1,1,0), (0,0,1,1), (1,0,0,1), (1,1,1,1))$

$MA - V = ((1,1,0,1) - 1, (0,0,1,1) - 0, (1,0,0,0) - 1, (1,1,1,1) - 0)$   
 $= ((1,1,0,0), (0,0,1,1), (0,1,1,1), (1,1,1,1))$

## 5 The Algorithm

**Input:** Text string  $T$  of length  $n$ , pattern string  $P$  of length  $m$  and error bound  $k$ .

**Output:** All occurrences of prefixes of  $T$  whose edit distances with  $P$  are smaller than or equal to  $k$ .

$$PV(0) = 1^m$$

$$MV(0) = 0^m$$

$$ex = 2^{\lceil \log_2(m+k) \rceil - 1} - 1 - k$$

For  $i = 1, 2, \dots, m$  do

$$bc_i = i + ex$$

For  $i = 1, 2, \dots, n$  do

{

$$PH(i) = MV(i-1) \text{ or } \neg(((EQ(i) \text{ and } PV(i-1)) + PV(i-1)) \text{ xor } PV(i-1)) \text{ or } EQ(i)) \text{ or } PV(i-1))$$

$$MH(i) = (((EQ(i) \text{ and } PV(i-1)) + PV(i-1)) \text{ xor } PV(i-1)) \text{ or } EQ(i)) \text{ and } PV(i-1)$$

$$BC = BC + PH(i) - MH(i)$$

If  $((CBC + 10^{m-1}) \text{ xor } CBC) \text{ and } CBC(m) = 1$   
 END

If  $(PH(i) \text{ and } 0^{m-1})(m) = 1$ , then

$$Score = Score + 1$$

Else If  $(MH(i) \text{ and } 0^{m-1})(m) = 1$ , then

$$Score = Score - 1$$

$$PH(i) = PH(i) \gg 1$$

$$MH(i) = MH(i) \gg 1$$

$$PV(i) = MH(i) \text{ or } (\neg EQ(i) \text{ and } \neg MV(i-1) \text{ and } \neg PH(i))$$

$$MV(i) = PH(i) \text{ and } (EQ(i) \text{ or } MV(i-1))$$

If  $Score \leq k$  return location  $i$  as an answer.  
 }

## 6 Conclusion

In this paper, our algorithm can ignore most unnecessary operations for checking and shifts the partial window soon.

## References

- [1] Amir, A., Keselman, D., Landau, G. M., Lewenstein, M., Lewenstein, N. and Rodeh, M., Text Indexing and Dictionary Matching with One Error, *Journal of Algorithms*, Vol. 37, 2000, pp. 309-325.
- [2] Galil, Z. and Park, Z., An improved

algorithm for approximate string matching, *SIAM journal on Computing*, 19(6), 1990, pp.989-999.

[3] Landau, G. and Vishkin, U., Fast parallel and serial approximate string matching, *Journal of Algorithms*, 10, 1989, pp.157-169.

[4] Lee, R. C. T., String Matching Algorithm, 2010.

[5] Introduction to the Design and Analysis of Algorithms, Lee, R. C. T., Tseng, S. S., Chang, R. C., and Tsai, Y. T., *McGraw-Hill Education*, 2005.

[6] Hyyro, H., Bit-parallel approximation string matching algorithms with transposition, *Journal of Discrete Algorithms*, Vol. 3, 2005, pp. 215-229.

[7] Huynh, T. N. D., Hon, W. K., Lam, T. W. and Sung, W. K., Approximate String Matching Using Compressed Suffix Arrays, *Theoretical Computer Science*, Vol. 352, 2006, pp.240-249.

[8] Holub, J., Melichar, B., Approximate String matching using factor automata, *Theoretical Computer Science*, Vol. 249, 2000.

[9] Hyyro, H. and Navarro, G., Bit-parallel Witnesses and their Application to Approximate String Matching, *Algorithmica*, Vol. 41, NO. 3, 2005, pp.203-231.

[10] Myer, G., A fast bit-vector algorithm for approximate string matching based on dynamic programming, *Journal of the ACM*, 46(3), 1999, pp.395-415.

[11] Navarro, G. and Baeza-Yates, R., Very fast and Simple approximate string matching, *Information Processing Letters*, Vol. 72, 1999, pp.65-70.

[12] Ou, C. S. and Lee, R. C. T., Bit-Parallel Operations to Investigate Properties of Logical Vectors by Logical Operations, 2010.

[13] Sellers, P. H., String Matching with Errors, *Journal of Algorithms*, Vol. 20, No. 1, 1980, pp.443-453.

[14] Sellers, P., The theory and computation of evolutionary distances: pattern recognition, *Journal of Algorithms*, 1, 1980, pp.359-373.

[15] Tarhio, J. and Ukkonen, E., Approximate Boyer-Moore String Matching, *SIAM Journal on Computing*, Vol. 22, No. 2, 1993, pp.243-260.

[16] Ukkonen, E., Finding Approximate patterns in strings. *Journal of Algorithms*, 6, 1985, pp.100-118.

[17] Wu, S. and Manber, U., Fast Text Searching: Allowing Errors, *Communications of the ACM*, Vol. 35, 1992, pp.83-91.

[18] Wu, S., Manber, U. and Myers, G., A sub-quadratic algorithm for approximate limited expression matching. *Algorithmica*, 15(1), 1996, pp.50-67.